

Implémentation from scratch d'un MLP et moteur d'autodifférenciation appliqués à MNIST

Jianye Shi, Hao Qian, Xiang Bian, Abdenmour Boulmis

M1 CHPS – Université Paris-Saclay
UVSQ 2025/2026

1^{er} janvier 2026

Déroulement

- Contexte et problématique (5 min)
- Architecture et implémentation (8 min)
- Résultats expérimentaux (10 min)
- Conclusion et perspectives (2 min)

Objectifs

- Implémenter **from scratch** un réseau de neurones (MLP) en C++
- Développer un moteur de différenciation automatique
- Appliquer au dataset MNIST (reconnaissance de chiffres manuscrits)
- Double focus : **Machine Learning + High Performance Computing**

Aspect ML

- Rétropropagation
- Optimisation (SGD)
- Convergence

Aspect HPC

- Optimisation bas niveau
- Parallélisme (OpenMP)
- BLAS, vectorisation

Le problème MNIST

- **Dataset classique** de classification
- Images 28×28 pixels (784 features)
- 10 classes (chiffres 0-9)
- 60,000 images d'entraînement
- 10,000 images de test

[Image MNIST]

Objectif : $\sim 98\%$ de précision

Notre approche : MLP simple mais complet

- Architecture : $784 \rightarrow 128 \rightarrow 10$
- Activation ReLU
- Loss CrossEntropy

Architecture du MLP

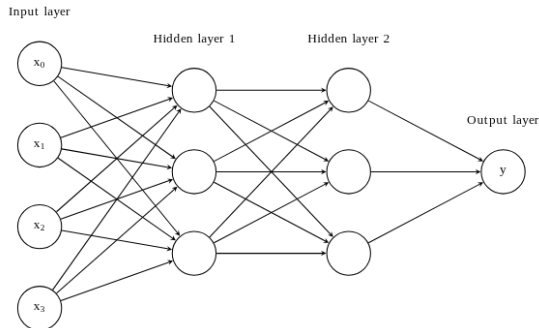


Figure – Structure hiérarchique du MLP

Propagation avant (Forward)

Entrée : $X \in \mathbb{R}^{N \times 784}$

Couche cachée :

$$H = \sigma(XW_1 + b_1)$$

Sortie :

$$Y = \text{softmax}(HW_2 + b_2)$$

Calcul par mini-batch

- Vectorisation des opérations
- Exploitation du parallélisme
- N exemples simultanément

Principe du DAG

- Chaque opération = nœud
- Stockage valeurs intermédiaires
- Calcul automatique gradients

Règle de la chaîne

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial x}$$

Backward pass

- Tri topologique (DFS)
- Propagation des gradients
- Mise à jour paramètres

Exemple simple

$$z = x \cdot w$$

$$y = z + b$$

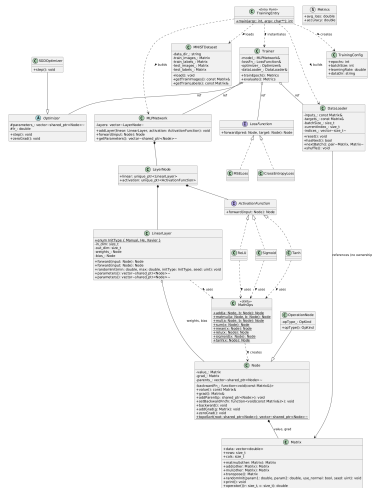
$$L = y^2$$

Gradients :

$$\frac{\partial L}{\partial y} = 2y$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial y}$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x$$



Architecture en couches : du calcul matriciel au pipeline d'entraînement

Construction du modèle MLPNetwork

Le réseau est construit par composition modulaire avec support multi-couches :

```
// Architecture flexible (network.cpp)
MLPNetwork MLPNetwork::createMultiHidden(
    int input_dim, const std::vector<int>& hidden_dims,
    int output_dim, const std::string& activation_name,
    const std::string& init_name, unsigned int seed
) {
    MLPNetwork net;
    int prev = input_dim;

    // Couches caches : prev -> h + activation
    for (int h : hidden_dims) {
        net.addLayer(
            makeLinear(prev, h, init_name, seed),
            makeActivation(activation_name)
        );
        prev = h;
    }

    // Couche de sortie : logits, sans activation
    net.addLayer(makeLinear(prev, output_dim, init_name, seed),
```


Choix de conception

- Stockage contigu : `std::vector<double>`
- Optimisation de la localité cache
- Support de la vectorisation (SIMD)

Implémentations `matmul`

- Naïve (*ijk*) : pédagogique
- Réordonnée (*ikj*) : cache-friendly
- Bloquée : optimisation cache
- OpenMP : parallélisation
- BLAS : référence optimisée

Sélection runtime

Initialisation des poids

- **He** : pour ReLU
 $\text{Var}(W) = 2/n_{in}$
- **Xavier** : pour Sigmoid/Tanh
 $\text{Var}(W) = 1/n_{in}$
- **Manual** : $[-0.1, 0.1]$

Reproductibilité

Graine contrôlée (`-seed`)

Moteur d'autodiff : Node et MathOps

Implémentation via Lambdas C++

- `std::shared_ptr<Node>` : gestion mémoire automatique
- Capture du contexte par closures
- Pas de classes dédiées par opération

```
// Exemple : Multiplication  $y = a * b$  (math_ops.cpp)
Node::Ptr mul(const Node::Ptr& a, const Node::Ptr& b) {
    Matrix out = a->value().mul(b->value());
    auto node = std::make_shared<OperationNode>(
        OpKind::MUL, out, {a, b}
    );

    node->setBackwardFn([a, b](const Matrix& grad) {
        a->addGrad(grad.mul(b->value())); //  $dL/da = grad * b$ 
        b->addGrad(grad.mul(a->value())); //  $dL/db = grad * a$ 
    });
    return node;
}
```

Pipeline d'entraînement

Modules principaux

- `LinearLayer` : $y = xW + b$
- `ActivationFunction`
ReLU, Sigmoid, Tanh
- `LossFunction`
MSE, CrossEntropy
- `Optimizer` : SGD
- `DataLoader` : mini-batch

Boucle d'entraînement

- 1 Charger mini-batch (X, Y)
- 2 **Forward** : $\hat{Y} = \text{Model}(X)$
- 3 **Loss** : $L = \text{Loss}(\hat{Y}, Y)$
- 4 **Backward** : calcul gradients
- 5 **Update** : $W \leftarrow W - \eta \nabla W$

```
// Implmentation SGD (optimizer.cpp)
void SGDoptimizer::step() {
    for (auto& p : parameters_) {
        Matrix& val = const_cast<Matrix&>(p->value());
        const Matrix& grad = p->grad();
        for(size_t i=0; i<val.data.size(); ++i) {
            val.data[i] -= lr_ * grad.data[i]; // W = W - W
        }
    }
}
```

Matériel

- AMD Ryzen 9 8940HX
- 16 cœurs / 32 threads
- 30 GiB RAM
- Cache L3 : 64 MiB

Logiciel

- Fedora Linux 42
- GCC 15.2.1
- Flags : `-O3 -march=native`

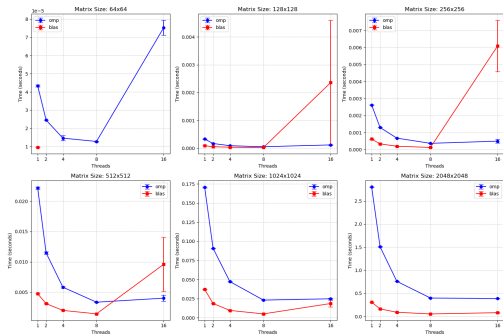
Protocole rigoureux

- **Fréquence verrouillée** : 4.0 GHz
- **Affinité CPU** : taskset
- **Warm-up** : 5 itérations
- **Graine fixée** : reproductibilité

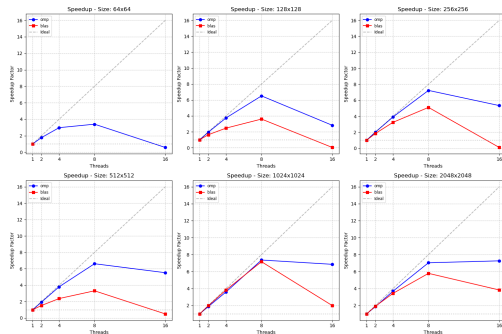
Objectif

Mesures fiables et reproductibles pour comparer les optimisations

Passage à l'échelle (Thread Scaling)



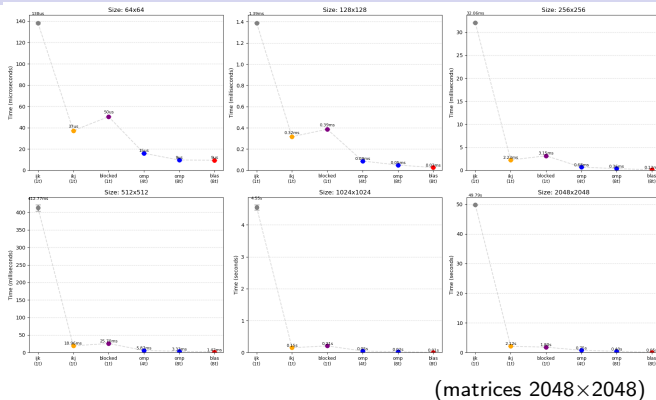
Temps d'exécution vs threads



Speedup vs threads

- **64x64** : surcoût > gain au-delà de 8 threads
- **2048x2048** : limité par bande passante mémoire
- **Optimum** : 8 threads = cœurs physiques

Hiérarchie des performances



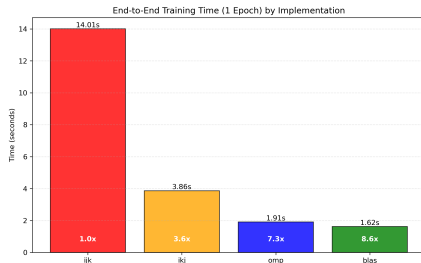
Version	Temps (ms)	Speedup	Technique
Naïf (<i>ijk</i>)	~3000	1×	Baseline
Réordonné (<i>ikj</i>)	~800	3.75×	Localité cache
OpenMP (8T)	~150	20×	Parallélisation

Avant optimisation (Naïf)

- matmul : **96%**
- Compute-bound
- Temps/époque : 13.27s

Après optimisation (BLAS)

- matmul : **<1%**
- I/O données : 43%
- Allocations : 29%
- Temps/époque : 1.74s



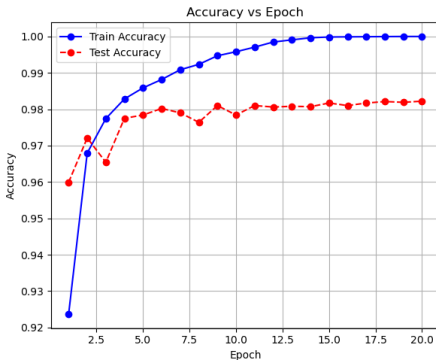
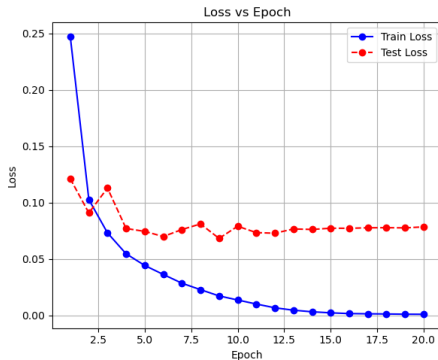
Répartition temps E2E

Speedup Global

7.6x

Limité par I/O (Amdahl)

Convergence : premiers résultats



LR=0.01 | Batch=64 | Hidden=128 | ReLU | 20 époques

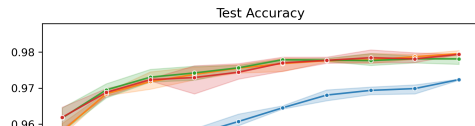
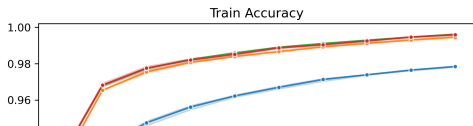
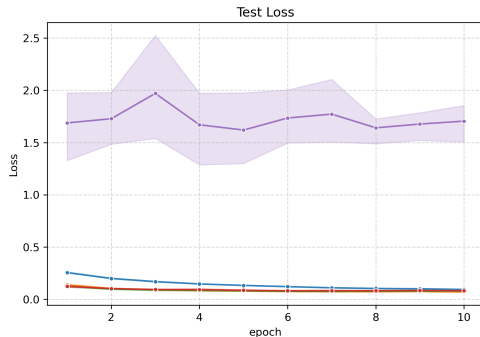
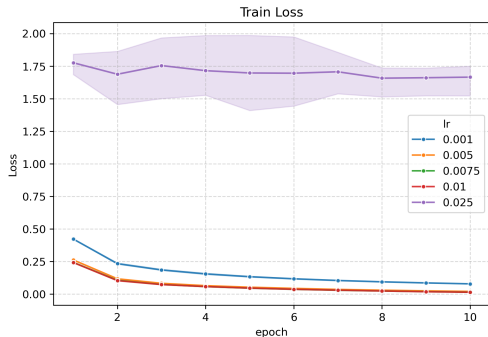
Précision
98.2%

Convergence
Rapide et stable

Généralisation
Écart modéré

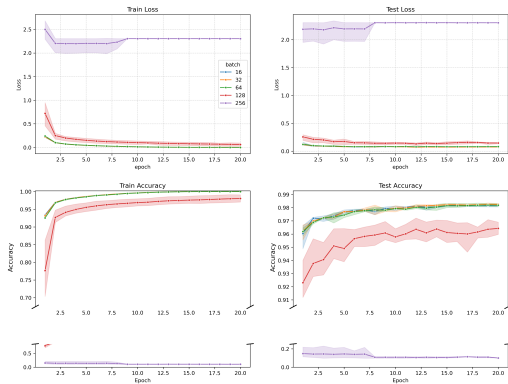
Impact du Learning Rate

Learning Rate Experiment Results (10 Epochs)



Impact du Batch Size

Batch Size Experiment Results (20 Epochs)



Influence de la taille de batch (20 époques, LR=0.01, 3 graines, Hidden=128)

B {16, 32}

Meilleur accuracy

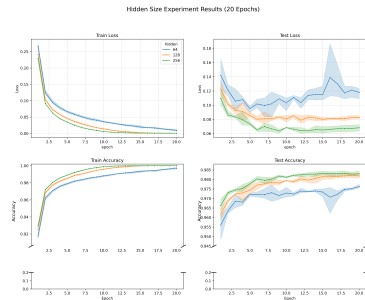
B = 64

Compromis optimal

B {128, 256}

Dégradation

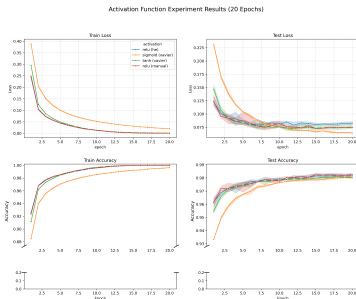
Impact de la taille cachée et de l'activation



Taille de couche cachée

Observations

- H=64 : Sous-apprentissage
- **H=128 : Optimal**
- H=256 : Gain marginal



Fonctions d'activation

Résultats

- **ReLU : Meilleur**
- Tanh/Sigmoid : OK (Xavier)

Configuration Finale

Learning Rate : 0.01

Batch Size : 64

Hidden Size : 128

Activation : ReLU

Initialisation : He

Précision validation : 98.2%

Répartition des tâches

- **État de l'art** : Jianye, Hao, Xiang (collectif)
- **Conception** : Jianye (documentation, UML)
- **Prototype MLP** : Abdenmour (tenseurs, couches, activations)
- **Moteur autodiff** : Hao + Xiang (graphe, opérations)
- **Intégration** : Jianye (pipeline complet, MNIST)
- **Entraînement** : Xiang (boucle), Jianye + Hao (optimizer, loss)
- **Optimisation HPC** : Jianye (profiling, benchmarks, Grid Search)
- **Rapport** : Collectif

Méthodologie

Développement incrémental avec validation continue à chaque étape

Réalisations

- Implémentation complète **from scratch** en C++
- Moteur d'autodiff fonctionnel et élégant (DAG + lambdas)
- Validation ML : **~98.2%** de précision sur MNIST
- Optimisation HPC : speedup **200×** sur le noyau de calcul

Enseignements clés

- **Dual challenge** : algorithmique (convergence) + computationnel (performance)
- **Loi d'Amdahl** : après optimisation du calcul, les I/O deviennent le goulot
- **Importance du protocole** : mesures reproductibles essentielles

Extensions ML

- Optimiseurs avancés (Momentum, Adam)
- Régularisation (L2, Dropout)
- Architectures CNN
- Datasets plus complexes

Optimisations HPC

- Support GPU (CUDA)
- Vectorisation explicite (AVX-512)
- Memory pool / arenas

Parallélisme à l'échelle

- Data parallelism (MPI)
- Model parallelism
- Pipeline de données optimisé

Reproductibilité

- Framework de benchmarking
- CI/CD pour validation
- Documentation utilisateur

Projet complet disponible sur [[lien GitHub](#)]

Implémentation du produit matriciel

```
// Version rordonne (ikj) - meilleure localit for (int i = 0
    i < m ++i) for (int k = 0 k < n ++k) for (int j = 0 j < p ++j) C(i,j) += A(i,k) * B(k,j)
// Version OpenMP pragma omp parallel for for (int i = 0 i < m ++i) for (int k = 0 k < n ++k) for
    (int j = 0 j < p ++j) C(i,j) += A(i,k) * B(k,j) ;;;;;;;;;;
```

Détails techniques : Backward pass

Calcul de gradient pour LinearLayer (layer.cpp)

Forward pass

$$Y = XW + b$$

$$X \in \mathbb{R}^{N \times d_{in}}$$

$$W \in \mathbb{R}^{d_{in} \times d_{out}}$$

$$b \in \mathbb{R}^{1 \times d_{out}}$$

Backward pass

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} \cdot W^T$$

$$\frac{\partial L}{\partial W} = X^T \cdot \frac{\partial L}{\partial Y}$$

$$\frac{\partial L}{\partial b} = \sum_i \frac{\partial L}{\partial Y_i}$$

Implémentation (math_ops.cpp)

- Stockage de X pendant forward
- Calcul des trois gradients
- Accumulation via `addGrad()`

Tri topologique (node.cpp)

- DFS récursif
- Ordre : parents $\rightarrow$ enfants
- Backward : ordre inverse
- Gradient initial = 1.0